COS30018 - Intelligent Systems

Task C.3: Data Processing 2

Stock Prediction System v0.3 - 10417506

Student: Le Minh Kha

1. Introduction

This report documents the implementation of Task C.3 for the COS30018 Intelligent Systems unit. Task C.3 focuses on advanced data visualization techniques for financial stock data, specifically implementing candlestick charts and boxplot charts to provide meaningful visual representations of stock price movements over time.

The implementation builds upon version 0.2 of the stock prediction system (from Task C.2), adding two new visualization functions while preserving all existing data loading and LSTM model functionality. The system continues to use CBA.AX (Commonwealth Bank of Australia) stock data from Yahoo Finance, spanning from January 2020 to July 2024.

Candlestick charts are essential tools in technical analysis, displaying four key price points (Open, High, Low, Close) for each time period. Boxplot charts provide statistical summaries showing the distribution of prices within defined time windows, including median, quartiles, and outliers.

2. Requirements Analysis

Task C.3 specifies two main implementation requirements:

2.1 Requirement 1: Candlestick Chart Function

The first requirement is to write a function that displays stock data using a candlestick chart. The function must use the mplfinance library and include an option where each candlestick can represent n trading days, where n is greater than or equal to 1. When n equals 1, each candle represents a single trading day (daily chart). When n is greater than 1, multiple trading days are aggregated into a single candle using standard OHLCV resampling rules.

2.2 Requirement 2: Boxplot Chart Function

The second requirement is to write a function that displays stock data using a boxplot chart for a moving window of n consecutive trading days. Each boxplot should summarize the distribution of stock prices within its respective time window, showing key statistics such as the median, quartiles (Q1 and Q3), and outliers.

2.3 Implementation Summary

To meet these requirements, two functions were implemented: plot_candlestick() for candlestick chart visualization and plot_boxplot() for boxplot visualization. The mplfinance library was added as a new dependency specifically for rendering professional-quality candlestick charts. Both functions support configurable time periods and date range filtering.

3. Code Explanation

This section provides detailed explanations of the non-trivial code segments in both visualization functions. The focus is on the logic and techniques that are not immediately obvious from the code itself.

3.1 Candlestick Chart Function (plot_candlestick)

3.1.1 Function Signature and Parameters

Function definition:

```
def plot_candlestick(df, n_days=1, ticker="Stock", start_date=None,
end_date=None):
```

The function accepts five parameters: df is the DataFrame containing stock data with OHLC (Open, High, Low, Close) columns; n_days controls how many trading days each candlestick represents (default is 1 for daily candles); ticker is the stock symbol used in the chart title; start_date and end_date filter the data to a specific date range. When n_days is set to 5, each candle represents approximately one trading week, and when set to 20, each candle represents approximately one trading month.

3.1.2 Column Name Standardization

A significant challenge in this implementation is that mplfinance requires specific column names in title case (Open, High, Low, Close, Volume), while our data from Yahoo Finance uses lowercase names. Additionally, our data contains both 'close' and 'adjclose' columns, and we must avoid creating duplicate 'Close' columns which would cause a type error.

Column mapping logic:

```
cols_lower = {col: col.lower() for col in data.columns}
ohlcv = {}

for col, col_lower in cols_lower.items():
    if col_lower == 'open':
        ohlcv['Open'] = data[col]
    elif col_lower == 'high':
        ohlcv['High'] = data[col]
    elif col_lower == 'low':
        ohlcv['Low'] = data[col]
    elif col_lower == 'volume':
        ohlcv['Volume'] = data[col]

# For Close: prefer adjclose over close
if 'adjclose' in cols_lower.values():
    adjclose_col = [c for c, cl in cols_lower.items() if cl == 'adjclose'][0]
    ohlcv['Close'] = data[adjclose_col]
elif 'close' in cols_lower.values():
    close_col = [c for c, cl in cols_lower.items() if cl == 'close'][0]
    ohlcv['Close'] = data[close_col]
```

This approach first creates a lowercase mapping of all column names, then selectively builds a new dictionary with properly named columns. The key insight is to prefer 'adjclose' (adjusted close) over 'close' for the Close column because adjusted close accounts for stock splits and dividends, providing more accurate historical price data. By building a new dictionary rather than renaming columns in place, we avoid the duplicate column issue entirely.

3.1.3 Data Resampling for Multi-Day Candles

When n_days is greater than 1, we need to aggregate multiple trading days into single candlesticks. This requires applying the correct aggregation function to each OHLCV column according to financial standards.

Resampling logic:

```
if n_days > 1:
    ohlc_dict = {
        'Open': 'first',    # First opening price in the period
        'High': 'max',      # Maximum high across the period
        'Low': 'min',       # Minimum low across the period
        'Close': 'last',    # Last closing price in the period
    }
    if 'Volume' in data.columns:
        ohlc_dict['Volume'] = 'sum'

    data = data.resample(f'{n_days}D').agg(ohlc_dict).dropna()
```

The resample() method groups data by time periods, similar to how groupby works but specifically for time series data. The frequency string (for example, '5D' means 5 calendar days) defines the grouping interval. The agg() method then applies different aggregation functions to different columns: 'first' takes the opening price of the first day, 'max' finds the highest price across all days, 'min' finds the lowest price, 'last' takes the closing price of the last day, and 'sum' totals the volume. The dropna() call removes any empty periods (such as weekends or holidays with no trading data).

3.1.4 Chart Style Configuration

Style configuration:

```
market_colors = mpf.make_marketcolors(
    up='green',       # Bullish candle (price increased)
    down='red',       # Bearish candle (price decreased)
    wick='inherit',   # Wick inherits from up/down
    edge='inherit',   # Edge inherits from up/down
    volume='in'       # Volume bars match candle direction
)

chart_style = mpf.make_mpf_style(
    marketcolors=market_colors,
    gridstyle='--',
    gridaxis='both'
)
```

The mplfinance library provides extensive customization through style objects. The make_marketcolors() function defines the color scheme: green for bullish candles (where the closing price is higher than the opening price, indicating the price went up) and red for bearish candles (where the closing price is lower than the opening, indicating the price went down). The 'inherit' setting for wicks and edges means they will use the same color as the candle body. The 'in' setting for volume makes volume bars match their corresponding candle colors.

3.2 Boxplot Chart Function (plot_boxplot)

3.2.1 Function Signature and Parameters

Function definition:

```
def plot_boxplot(df, n_days=5, column='adjclose', ticker="Stock",
                 start_date=None, end_date=None):
```

The function takes six parameters: df is the DataFrame containing stock data; n_days specifies the window size (how many consecutive trading days per boxplot); column determines which data column to visualize (default is 'adjclose'); ticker is the stock symbol for the title; and start_date/end_date filter the date range. Unlike the candlestick function which must work with OHLCV data, this function can plot any numeric column.

3.2.2 Window Division Algorithm

The core logic divides the data into non-overlapping windows of n consecutive trading days. Each window becomes one boxplot showing the distribution of values within that period.

Window division logic:

```
window_data = []
window_labels = []

for i in range(0, len(data), n_days):
    window = data.iloc[i:i + n_days]

    if len(window) < n_days:
        break

    window_data.append(window[column].values)

    label_start = window.index[0].strftime('%Y-%m-%d')
    label_end = window.index[-1].strftime('%Y-%m-%d')
    window_labels.append(f"{label_start}\nto\n{label_end}")
```

The range() function with step n_days generates start indices: 0, n_days, 2*n_days, and so on. For each start index, iloc[i:i+n_days] selects n_days consecutive rows. The incomplete window check ensures we do not create misleading boxplots from insufficient data at the end of the dataset. The .values property converts the pandas Series to a numpy array, which is what matplotlib's boxplot function expects. The strftime('%Y-%m-%d') method formats datetime objects into readable date strings for the x-axis labels.

3.2.3 Boxplot Styling

Boxplot styling configuration:

```
bp = ax.boxplot(
    window_data,
    labels=window_labels,
    patch_artist=True,
    boxprops=dict(facecolor='lightblue', edgecolor='black', linewidth=1.2),
    medianprops=dict(color='red', linewidth=2),
    whiskerprops=dict(color='black', linewidth=1.0),
    capprops=dict(color='black', linewidth=1.0),
    flierprops=dict(marker='o', markerfacecolor='gray', markersize=4,
alpha=0.6),
    widths=0.6
)
```

The patch_artist=True parameter is essential for filling box bodies with color (by default, boxes are hollow outlines). Each styling parameter controls a different component: boxprops styles the box body itself, medianprops styles the red horizontal line showing the median value, whiskerprops styles the vertical lines extending to minimum and maximum values, capprops styles the horizontal caps at whisker ends, and flierprops styles outlier points (values beyond 1.5 times the interquartile range from Q1 or Q3). The alpha parameter for fliers controls transparency, making outliers less visually dominant.

4. Execution Results

This section presents the visual outputs generated by executing stock_prediction_v03.py with CBA.AX stock data from January 2020 to July 2024.

4.1 Candlestick Charts

4.1.1 Daily Candlestick Chart (n_days=1)

The daily candlestick chart shows individual trading days from January 2024 to July 2024 (a 6-month period for readability). Each candle represents one trading day's price action.

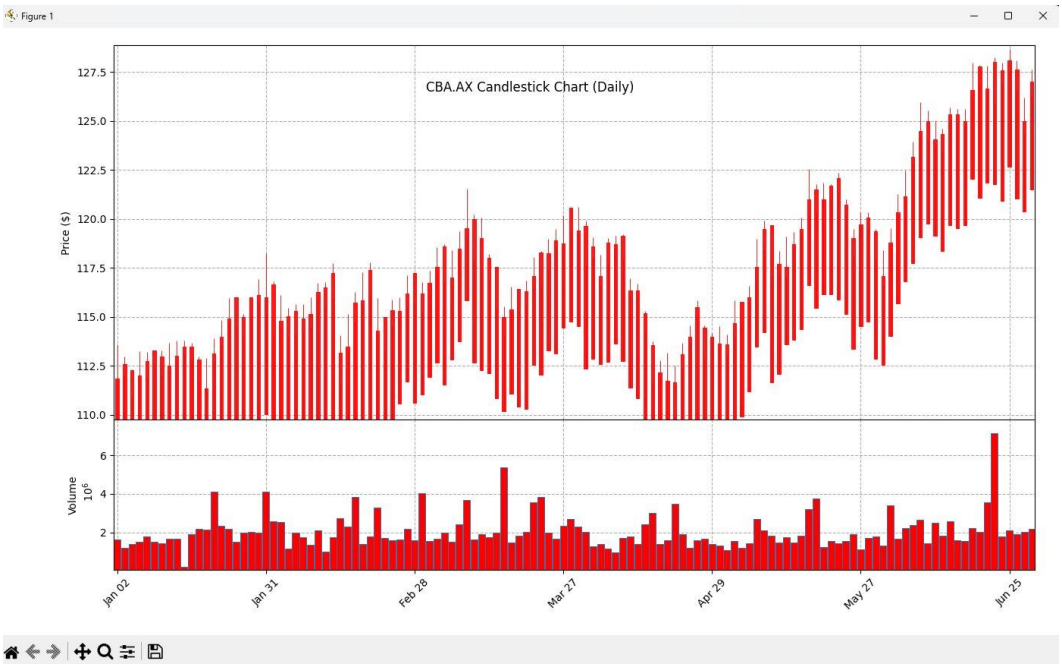

Figure 1: Daily Candlestick Chart (January-June 2024)

4.1.2 Weekly Candlestick Chart (n_days=5)

The weekly candlestick chart aggregates 5 trading days into each candle, providing a broader view of price trends across the full 4.5-year dataset. Volume bars below show weekly trading activity.

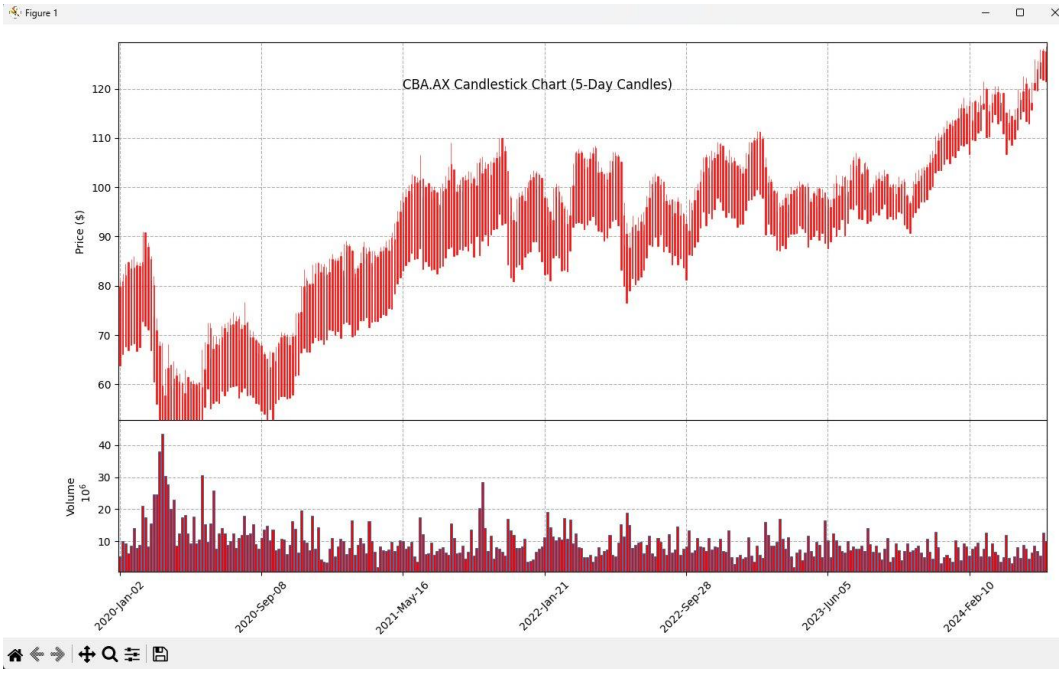

Figure 2: 5-Day (Weekly) Candlestick Chart (2020-2024)

4.1.3 Monthly Candlestick Chart (n_days=20)

The monthly candlestick chart aggregates 20 trading days (approximately one calendar month) into each candle. This provides the highest-level view of long-term price movements, clearly showing the COVID-19 crash in early 2020 and subsequent recovery.

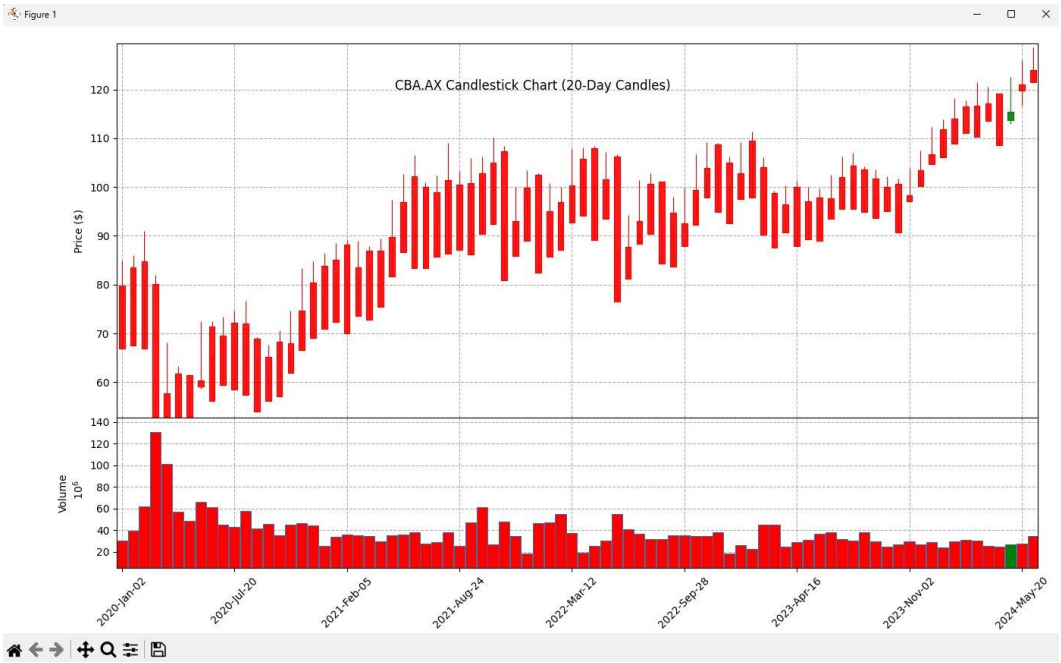

Figure 3: 20-Day (Monthly) Candlestick Chart (2020-2024)

4.2 Boxplot Charts

4.2.1 5-Day Window Boxplot

The 5-day boxplot shows the distribution of adjusted closing prices for each trading week from January 2024 to July 2024. Each box represents the price distribution within one week, with the red line indicating the median price.

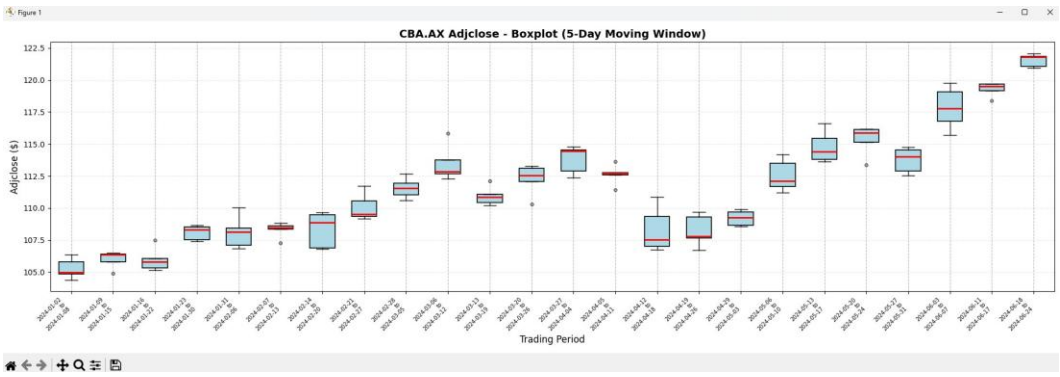

Figure 4: 5-Day Window Boxplot (January-June 2024)

4.2.2 20-Day Window Boxplot

The 20-day boxplot shows monthly price distributions across the full dataset. This visualization reveals how price volatility (indicated by box height) varies over time, with notably higher volatility during the COVID-19 period in early 2020.

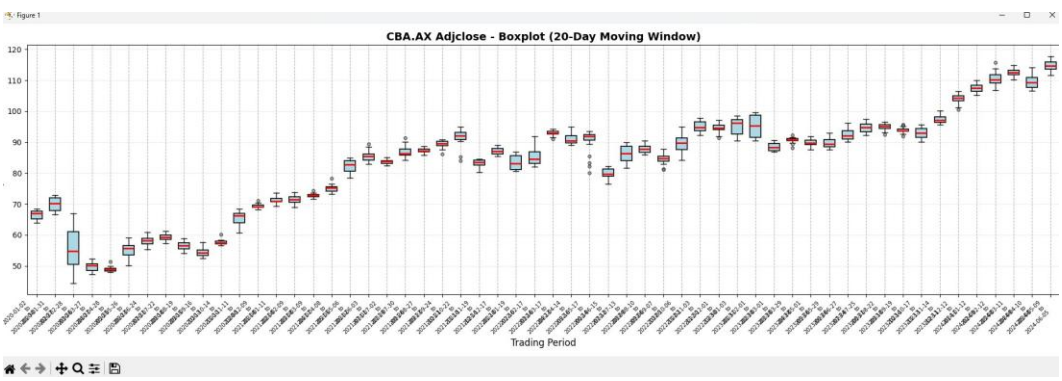

Figure 5: 20-Day Window Boxplot (2020-2024)

4.3 Model Training Results

The model training and evaluation was performed alongside the visualization tasks. The console output shows performance metrics, and the results plot shows the training loss progression and prediction accuracy.

```
Epoch 22/25
28/28 ━━━━━━━━━━━━━━━━━━━━ 4s 148ms/step - loss: 0.0286 - mean_absolute_error: 0.0286 - val_loss: 0.0200 - val_mean_absolute_error: 0.0200
Epoch 23/25
28/28 ━━━━━━━━━━━━━━━━━━━━ 4s 148ms/step - loss: 0.0285 - mean_absolute_error: 0.0285 - val_loss: 0.0695 - val_mean_absolute_error: 0.0695
Epoch 24/25
28/28 ━━━━━━━━━━━━━━━━━━━━ 4s 145ms/step - loss: 0.0286 - mean_absolute_error: 0.0286 - val_loss: 0.0733 - val_mean_absolute_error: 0.0733
Epoch 25/25
28/28 ━━━━━━━━━━━━━━━━━━━━ 4s 144ms/step - loss: 0.0305 - mean_absolute_error: 0.0305 - val_loss: 0.0530 - val_mean_absolute_error: 0.0530

============================================================
EVALUATING MODEL
============================================================
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 132ms/step

PERFORMANCE METRICS - v0.3
============================================================
Mean Absolute Error (MAE):      $4.14
Root Mean Squared Error (RMSE): $4.61
Mean Squared Error (MSE):       21.22
Mean Absolute Percentage Error: 3.84%
R² Score:                       0.7348
============================================================

Results saved to 'v03_results.png'

============================================================
FUTURE PREDICTION
============================================================
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step
Predicted price for next trading day: $114.28
```

Figure 6: Console Output Showing Performance Metrics

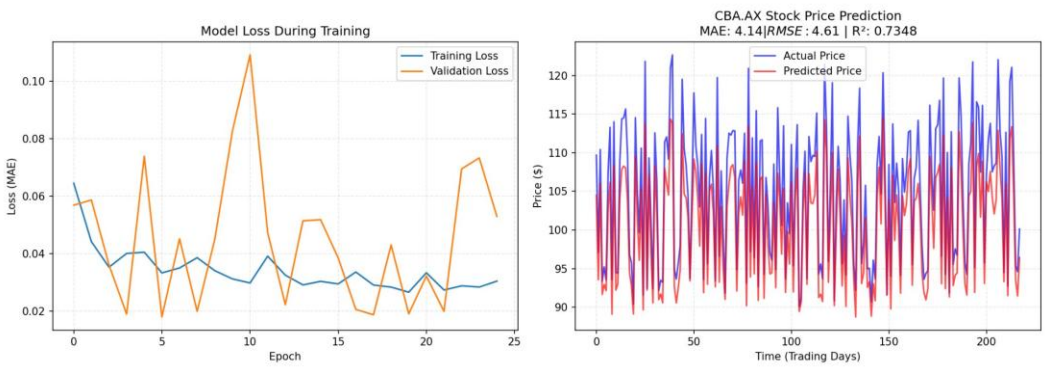

Figure 7: Training Loss and Stock Price Prediction Results

Performance Metrics Summary:

Metric	Value
Mean Absolute Error (MAE)	$4.14
Root Mean Squared Error (RMSE)	$4.61
Mean Absolute Percentage Error (MAPE)	3.84%
R^2 Score	0.7348

5. Challenges and Solutions

Several challenges were encountered during the implementation of Task C.3. This section documents the main issues and the solutions applied.

5.1 Column Name Standardization

The mplfinance library requires specific column names in title case (Open, High, Low, Close, Volume), but Yahoo Finance data uses lowercase names. Additionally, our DataFrame contains both 'close' and 'adjclose' columns. A naive approach of renaming all columns to title case would create duplicate 'Close' columns, causing a type error. The solution was to build a new DataFrame by selectively mapping columns and preferring 'adjclose' over 'close' for the Close column, as adjusted close is more accurate for historical analysis.

5.2 Date Range Filtering

For daily candlestick charts with the full 4.5-year dataset, the chart became too dense to read. The solution was to use a shorter date range (January to July 2024) for daily charts while showing the full range for aggregated charts (5-day and 20-day), where the data density is more manageable.

5.3 Incomplete Windows in Boxplot

When dividing data into fixed-size windows, the final window often has fewer data points than n_days. Including this incomplete window would create a misleading boxplot with different sample size. The solution was to check window length and skip any windows that have fewer than n_days data points, ensuring all boxplots represent comparable time periods.

5.4 X-Axis Label Readability

For boxplot charts with many windows, the x-axis date labels overlapped and became unreadable. The solution involved rotating labels by 45 degrees, using multi-line labels (start date, 'to', end date), and reducing font size. Additionally, the figure width was made dynamic, scaling with the number of boxplots while bounded by minimum and maximum values.

6. References

Candlestick Chart Implementation: https://coderzcolumn.com/tutorials/data-science/candlestick-chart-in-python-mplfinance-plotly-bokeh

mplfinance Library Documentation: https://github.com/matplotlib/mplfinance

Pandas resample() Method: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.resample.html

Matplotlib Boxplot Documentation: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.boxplot.html

7. Conclusion

Task C.3 has been successfully completed with the implementation of two visualization functions for stock market data analysis. The plot_candlestick() function provides flexible candlestick chart generation using the mplfinance library, supporting variable time aggregation from daily to monthly views. The plot_boxplot() function offers statistical distribution visualization through boxplots with configurable window sizes.

Both functions include proper error handling, data validation, and extensive documentation. The implementation successfully handles the challenges of column name standardization, date range filtering, and visual readability. The code follows the pattern established in versions 0.1 and 0.2, maintaining compatibility with the existing stock prediction system while adding new visualization capabilities.

The visualization results demonstrate the system's ability to present CBA.AX stock data in multiple formats, providing both detailed daily views and aggregated long-term perspectives. These visualizations complement the LSTM-based prediction model, offering traders and analysts multiple ways to understand historical price behavior and model performance.